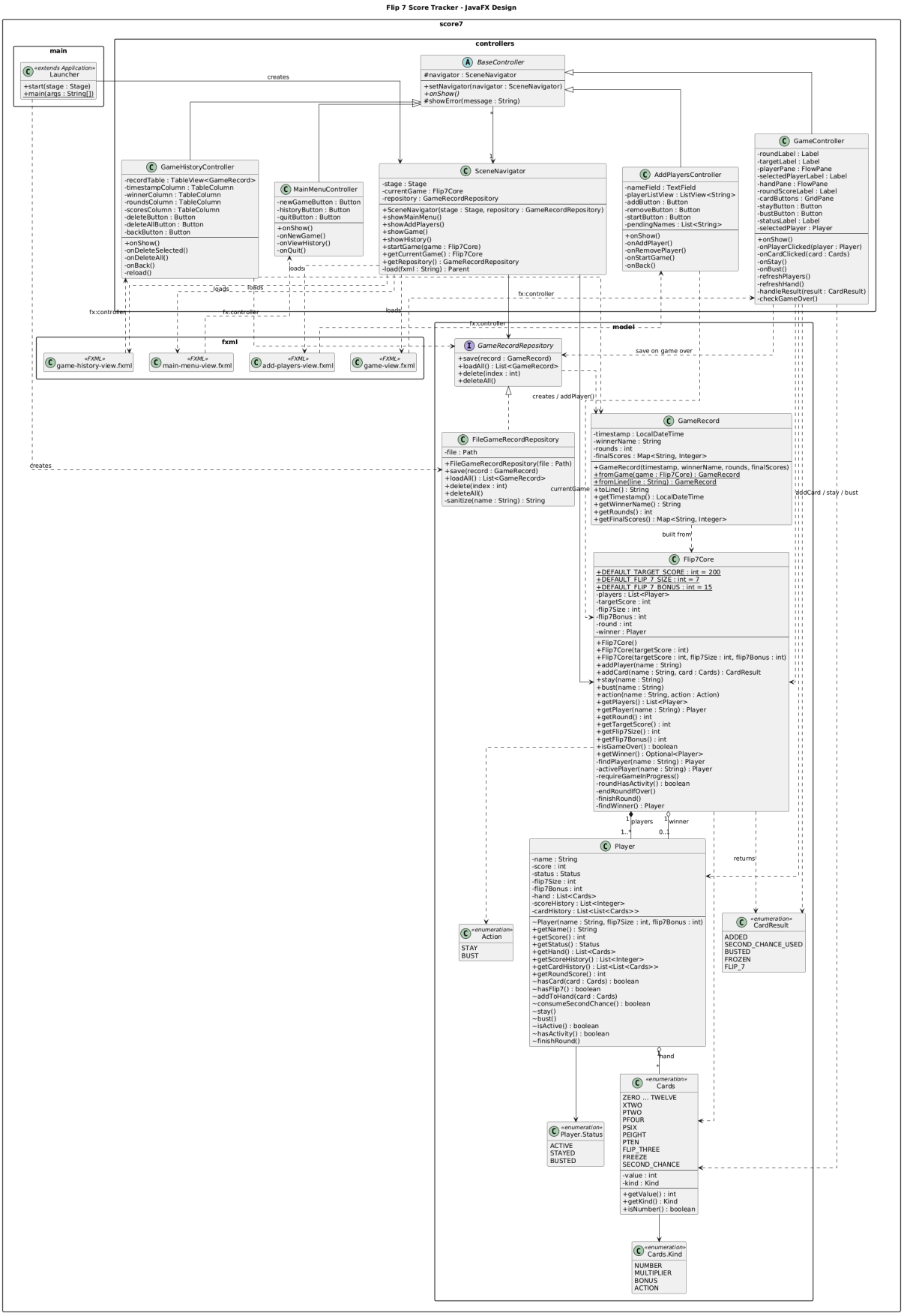


Figure 1: Made with https://editor.plantuml.com/

The diagram above shows the planned structure of the application. Some details may change during implementation, but this is how it would work now. It is likely that at implementation time some of these classes will be simplified or entirely omitted.

`Launcher.java` starts the application. It creates the game record repository and a `SceneNavigator`, which is responsible for keeping track of the current game and switching between scenes as required.

Each scene has its own controller (`MainMenuController`, `AddPlayersController`, `GameController`, and `GameHistoryController`) that contains the code for making the buttons in that scene work. They all extend `BaseController`, which gives them access to the `SceneNavigator`. When a controller needs to change scenes or read the current game, it asks the navigator.

The game logic lives in the `model` package, separate from the JavaFX code. `Flip7Core` tracks the players, rounds, and winner, and finished games are saved through the `GameRecordRepository` interface.

This UML diagram is based on a console prototype of the app, built in the style of Option 2. The `model` package (`Flip7Core`, `Player`, `Cards`, `CardResult`, and `Action`) comes directly from that prototype. The record-saving classes (`GameRecord`, `GameRecordRepository`, and `FileGameRecordRepository`) are new. In the prototype, that logic lives in `Main.java`, and it was separated out for the JavaFX version. The controllers, `SceneNavigator`, and FXML views are also new, since they replace the console menus in `Main.java`.